PROJECT COMPREHENSIVE TECHNICAL GUIDE

CareerCompass AI

AI-Powered Career Recommendation Platform for Engineering Students

Complete Technical Architecture, Machine Learning Implementation, System Integration, and Viva Preparation Manual

---

Document Purpose: This guide provides an end-to-end, beginner-friendly yet technically thorough breakdown of the CareerCompass AI platform. It explains the exact technologies used, how the Scikit-Learn Random Forest Classifier was trained and deployed via FastAPI, how the React frontend interfaces with the Express backend, how personalized learning roadmaps are constructed, and answers critical examination and viva questions.

Author / Development Team: Full-Stack & Machine Learning Engineering Team
Project Domain: Educational Technology (EdTech) / Artificial Intelligence / Career Trajectory Guidance
Software Version: 2.0.0 (Production Architecture with Bcrypt Authentication & REST API Isolation)
Primary Web URL: http://localhost:5000 | **ML API Service:** http://127.0.0.1:8000
Date of Documentation: October 2026

Core Machine Learning:	Scikit-Learn RandomForestClassifier (200 estimators) trained on 550 multi-disciplinary engineering profiles across 11 career classes.
Web Stack Architecture:	React 19 + TypeScript + Vite + Tailwind CSS frontend coupled to an Express.js API gateway and a Python FastAPI ML microservice.
Security & User Storage:	JSON Web Tokens (JWT) + Bcrypt password hashing (10 salt rounds) with isolated, per-student profile and progress persistence.
Adaptive Roadmaps:	Dynamic 4-phase milestone engine computing verified skill overlap vs. high-priority gaps, generating customized project deliverables.

Table of Contents

Section 1	Project Overview & Problem Statement	Page 3
Section 2	Complete Technology Stack Breakdown	Page 4
Section 3	System Architecture & Inter-Component Flow	Page 5
Section 4	Step-by-Step User Journey	Page 6
Section 5	Dataset Analysis & Feature Engineering	Page 7
Section 6	Random Forest Classifier: Principles & Hyperparameters	Page 8
Section 7	Where Exactly Machine Learning Happens in Code	Page 10
Section 8	Where AI, ML & Rule-Based Engines Are Applied	Page 11
Section 9	Backend Architecture & REST API Endpoints	Page 12
Section 10	Model Evaluation Results & Synthetic Data Caveats	Page 13
Section 11	Skill-Gap Analysis & Adaptive Roadmap Generation	Page 14
Section 12	Authentication System & Data Isolation	Page 15
Section 13	Project File Structure & Key Code Map	Page 16
Section 14	How to Run the Application Locally	Page 17
Section 15	Common Errors, Diagnosis & Troubleshooting	Page 18
Section 16	Project Limitations & Future Scope	Page 19
Section 17	Comprehensive Viva & Presentation Preparation (25 Questions)	Page 20
Section 18	Glossary of Key Technical Terms	Page 23
Section 19	Final Quick-Revision Sheet & Architecture Summary	Page 24

1. Project Overview & Problem Statement

1.1 What is CareerCompass AI?

CareerCompass AI is an intelligent, explainable career discovery and guidance platform built specifically for engineering college students. It solves one of the most common dilemmas in higher education: students learn multiple programming languages, libraries, and mathematical concepts during their curriculum, but struggle to know which modern technical industry domain best suits their existing capabilities and what exact steps they must take to become career-ready.

1.2 The Problem It Solves (With Concrete Example)

Consider a 2nd or 3rd-year Computer Science Engineering student named *Aarav*. Aarav has learned Python in college, knows basic SQL queries, and has built a couple of basic web pages using HTML and CSS. Aarav asks himself: *'Should I pursue Full-Stack Software Development, Data Analytics, or Artificial Intelligence? Which one fits my current strengths? What skills am I still missing? What project should I build for my portfolio?'*

Without guidance, students often pick random online courses, become overwhelmed by prerequisite jargon, or graduate without coherent domain projects. CareerCompass AI solves this by taking the student's exact branch, current year, skill proficiencies, interests, and working styles, feeding them into a trained Machine Learning model combined with a multi-factor compatibility engine, and returning:

- **Top 3 Ranked Career Domains:** For example: #1 Software Development (84% match), #2 Data Analytics & BI (76% match), #3 AI & Machine Learning (71% match).
- **Clear Explainability:** Explicit rationale showing which existing skills matched the domain and why their branch synergy supports this trajectory.
- **Skill-Gap Identification:** Exact missing libraries or tools needed (e.g. Git, React, Docker for Software; Power BI and Statistics for Data Analytics).
- **Adaptive Phased Roadmap:** A customized 4-phase learning schedule with concrete portfolio project deliverables and weekly hour estimates.

1.3 Supported Engineering Disciplines

Unlike generic software job portals, CareerCompass AI was architected from day one to embrace core and interdisciplinary engineering branches, including: Computer Engineering / CSE, Information Technology (IT), Electronics and Communication (ECE / VLSI), Electrical & Renewable Energy, Mechanical Design & CAD, Civil & Infrastructure (BIM), and Chemical & Process Engineering. Cross-branch skill pivots (e.g. an Electrical student pivoting to Data Analytics) are natively modeled.

1.4 Scope & Boundary Definitions

To maintain professional clarity and focus, CareerCompass AI does not operate as an open social network, general-purpose course marketplace, or job board. The current verified release focuses exclusively on **Career Discovery + Explainable Recommendations + Adaptive Learning Roadmaps + Domain Bookmarking**.

2. Complete Technology Stack Breakdown

The following table documents every technology, library, and framework verified in the active codebase:

Layer	Technology	Where Used	Why & How It Contributes
Frontend	**React 19**	client/src/App.tsx, components	Powers the interactive single-page application (SPA), managing view transitions, assessment wizard steps, and responsive reactive dashboards.
Frontend	**TypeScript**	client/src/types/	Enforces strict compile-time types for candidate profiles, recommendation payloads, roadmap milestones, and authentication states.
Frontend	**Tailwind CSS v4**	All UI component classes	Provides a utility-first, modern SaaS visual design system with deep navy typography, restrained blue accents, and subtle borders.
Frontend	**Vite v8**	client/vite.config.ts	Fast build tool and development server, bundling production code into minified JavaScript/CSS assets in client/dist.
Backend	**Node.js & Express 5**	server/server.js (Port 5000)	Serves as the primary application gateway, handling HTTP requests, client bundle hosting, catalog lookups, and orchestrating ML calls.
Backend	**Bcryptjs & JWT**	server/auth.js	Secures user registration with salted password hashing (10 rounds) and validates requests via signed JSON Web Tokens (7-day validity).
ML Service	**FastAPI & Uvicorn**	ml/app.py (Port 8000)	Asynchronous high-performance Python microservice that exposes the trained Random Forest model over REST endpoints (/predict).
Machine Learning	**Scikit-Learn**	ml/train_model.py	Implements the RandomForestClassifier pipeline, ColumnTransformer, OneHotEncoder, and cross-validation.
Data Processing	**Pandas & NumPy**	ml/preprocessing.py	Handles tabular dataset loading, vector multi-hot skill encoding, missing value imputation, and matrix array transformations.
Serialization	**Joblib**	ml/artifacts/*.joblib	Serializes the entire scikit-learn pipeline (preprocessor + model weights) to disk and reloads it instantly in memory during startup.
Database Layer	**Persistent JSON Store**	server/data/	File-backed database system storing user accounts (users.json), isolated profiles (saved_profiles.json), and domains.

3. System Architecture & Inter-Component Flow

CareerCompass AI utilizes a modern, decoupled microservice architecture consisting of three primary layers: the React single-page frontend, the Express API gateway, and the FastAPI ML inference engine.

```
+-------------------------------------------------------------------------------------+ | STUDENT USER |
+-------------------------------------------------------------------------------------+ | (HTTPS / Browser) v
+-------------------------------------------------------------------------------------+ | FRONTEND
APPLICATION: React 19 + TypeScript (Port 5000) | | - AuthPage: Registration, Login & JWT Handling | | -
AssessmentWizard: Academic Year, Branch, Multi-Select Skills, Interests | | - Dashboard: Recommendation Cards,
Match Scores, Radar Chart, Roadmap Checklists |
+-------------------------------------------------------------------------------------+ | (HTTP JSON with
Bearer Token) v +-------------------------------------------------------------------------------------+ |
EXPRESS.JS BACKEND & API GATEWAY (Port 5000) | | - Auth Controller: Bcrypt Password Hashing & JWT Verification
(server/auth.js) | | - Recommendation Engine: Multi-Factor Compatibility Calculator | | - Adaptive Roadmap Engine:
Dynamic Phase & Task Generator | | - User Data Storage: Isolated persistence in server/data/saved_profiles.json |
+-------------------------------------------------------------------------------------+ | (Internal REST Call:
POST /predict) v +-------------------------------------------------------------------------------------+ |
FASTAPI PYTHON ML MICROSERVICE (Port 8000) | | - Pydantic Request Validation: StudentProfileRequest (ml/app.py) |
| - Custom SkillMultiHotEncoder: Multi-hot binary vector conversion | | - OneHotEncoder: Categorical branch, year,
interest transform | | - RandomForestClassifier Pipeline: Predicts class probabilities (predict_proba) | | -
Explainability Generator: Extracts top matching skills & high-priority gaps |
+-------------------------------------------------------------------------------------+
```

3.1 Component Communication Mechanics

1. Frontend to Express: When a student submits their profile, the React client bundles their skills, proficiencies, branch, and preferences into a JSON payload and dispatches a POST /api/recommend request containing the authenticated JWT in the Authorization: Bearer <token> header.

2. Express to FastAPI: The Express server validates the token, extracts the authenticated userId, and forward-proxies the candidate profile to the FastAPI service at http://127.0.0.1:8000/api/career-recommendations via server/mlClient.js with a 4-second timeout and fallback handling.

3. ML Inference & Explainability: FastAPI executes the Scikit-Learn pipeline. The SkillMultiHotEncoder converts student skills into a sparse binary vector. The classifier computes probabilities across all 11 domains. The top domains and feature explanations are returned as JSON.

4. Fusion & Roadmap Generation: Express merges the ML prediction confidence with multi-factor heuristic checks (branch synergy, academic year, work style), executes the adaptive roadmap generator, and persists the combined result under saved_profiles[userId] before returning it to the React dashboard.

4. Step-by-Step User Journey

The following 15-stage sequence details what happens at every step of the student's interaction:

Step 1: Visitor Arrival	Student navigates to http://localhost:5000 in their browser. The client checks local storage for an active session token. If no session exists, the professional authentication screen is displayed.
Step 2: Sign Up or Login	The student enters their name, email, and password. The frontend validates password strength (>=6 chars, alphanumeric) and calls POST /api/auth/register. The backend hashes the password with bcrypt, stores the account in users.json, and returns a signed JWT.
Step 3: Branch & Year Selection	New students enter the Assessment Wizard. In Step 1, they pick their discipline (e.g. Computer Science, ECE, Mechanical) and academic year (1st to 4th Year).
Step 4: Technical Skills Input	In Step 2, students select technical tools and concepts from dynamic branch-aware chip suggestions (e.g. Python, SQL, C++, AutoCAD, Verilog) or type custom skills.
Step 5: Skill Proficiency Rating	Students assign realistic mastery tiers to each chosen skill: Beginner, Intermediate, or Proficient.
Step 6: Interests & Working Style	In Step 3, students identify primary areas of curiosity (e.g. 'building applications', 'working with data', 'sensors and connected devices') and work style preferences ('hands-on building', 'analytical problem-solving').
Step 7: Career Aspirations & Commitment	Students specify career horizon goals ('internship first', 'industry job after graduation', 'research') and estimated weekly study hours (e.g. 10-15 hrs/week).
Step 8: Assessment Submission	Clicking 'Discover My Career Domain' triggers validation and sends the student profile payload to POST /api/recommend with the Bearer token.
Step 9: Express Validation & Dispatch	The backend authenticates the token, enforces req.user.name, and makes an asynchronous REST request to the FastAPI ML service.
Step 10: Machine Learning Classification	FastAPI's Random Forest classifier pipeline evaluates the profile and outputs predicted career domain probabilities (predict_proba).
Step 11: Multi-Factor Compatibility Scoring	The recommendation engine blends the ML probability score with branch synergy, work style fit, and goal timelines into a normalized composite match percentage (38% to 98%).
Step 12: Skill-Gap Computation	The engine runs a set-difference algorithm comparing candidate skills against the domain's core skill catalog, classifying skills into 'Matched Skills', 'Skills to Improve', and 'High-Priority Gaps'.
Step 13: Dynamic Roadmap Assembly	The adaptive roadmap generator drafts 4 sequential phases with custom milestone tasks, week durations calculated from the student's study hours, and capstone project deliverables.
Step 14: Isolated Profile Persistence	The complete profile, recommendations, and initial roadmap progress are saved under server/data/saved_profiles.json keyed strictly by the student's userId.
Step 15: Interactive Dashboard Rendering	The student explores their personalized dashboard: top 3 domain cards, skill radar comparison chart, clickable roadmap checklists, and domain bookmarking.

5. Dataset Analysis & Feature Engineering

5.1 Dataset Overview

The machine learning model was trained using the verified Excel workbook: Engineering_Career_Recommendation_Dataset_550.xlsx (Sheet name: Student_Profile_Dataset). The dataset consists of exactly **550 synthetic engineering student profiles** across 21 columns.

5.2 Input Features vs. Target Variable

To ensure robust machine learning without data leakage, the columns are cleanly split into inputs, targets, and excluded post-recommendation attributes:

Category	Column Names	Data Type & Handling
Target Variable	Recommended_Domain	Categorical label representing the ground-truth career classification. 11 distinct classes.
Skill Input	Current_Skills	Comma-separated string of technical skills (e.g. *'Python, SQL, HTML/CSS'*). Transformed via custom Multi-Hot binary vector encoding.
Categorical Inputs (8 Features)	Student_Branch, Study_Year, Primary_Interest, Work_Style_Preference, Personality_Traits, Academic_Performance, Certifications, Career_Aspiration	Nominal and ordinal student attributes. Encoded using OneHotEncoder(handle_unknown='ignore') into orthogonal binary columns.
Excluded Columns (Leakage Prevention)	Profile_ID, Suggested_Career_Path, Roadmap_Step_1..5, Portfolio_Project_Ideas, Potential_Job_Opportunities, Dataset_Note	Explicitly removed prior to training. These columns contain downstream outcomes derived from the target domain, which would cause artificial 100% data leakage.

5.3 Clarification: 11 Dataset Classes vs. 13 Platform Catalog Domains

A critical observation in the codebase is that while the platform catalog (server/data/career_domains.json) supports **13 engineering domains**, the Excel dataset contains exactly **11 distinct target classes** in Recommended_Domain. Two newer disciplines—*Chemical & Process Engineering* and *AR/VR & Game Development*—were defined in the platform catalog with domain metadata and roadmaps, but have no synthetic training examples in the 550-row workbook. For these 2 domains, the platform seamlessly utilizes multi-factor heuristic scoring as a verified fallback, while the Random Forest classifier actively predicts probabilities across the core 11 classes.

5.4 Synthetic Data Nature & Educational Utility

Important Caveat: *The 550 student profiles are synthetic—generated using structured rules to simulate realistic student profiles. While ideal for developing, verifying, and testing the end-to-end machine learning pipeline, high accuracy on synthetic data does not constitute clinical or empirical validation of human career outcomes. It serves as an experimental prototype.*

6. Random Forest Classifier: Detailed Explanation

6.1 Machine Learning Fundamentals

Machine Learning (ML) is a branch of Artificial Intelligence where computer systems learn patterns from historical examples rather than following manually hardcoded decision rules. In **Supervised Classification**, the algorithm is provided with input feature vectors *X* (a student's skills, branch, interests) and corresponding ground-truth target labels *y* (the recommended domain). The model's objective is to learn a mathematical mapping function $f(X) \rightarrow y$ that accurately predicts labels for unseen student profiles.

6.2 From Decision Tree to Random Forest Ensemble

A **Decision Tree** is an intuitive classifier that splits data by asking a series of binary questions (e.g. *'Does the student know Python? Yes/No'* → *'Is primary interest data analytics? Yes/No'*). While simple, individual decision trees suffer from high variance and easily *overfit* (memorize training noise).

A **Random Forest** overcomes this limitation through **Ensemble Learning** using *Bootstrap Aggregating (Bagging)*. Instead of building one tree, it constructs an ensemble of hundreds of diverse decision trees (here, 200 trees). Each tree is trained on a random sample of student profiles and considers a random subset of features at each split. When predicting, all 200 trees cast independent votes. The class with the majority vote wins, and the fraction of trees voting for each domain represents its **class probability**.

6.3 Model Configuration & Hyperparameters in Code

The model is instantiated in ml/train_model.py with the following production hyperparameters:

```
classifier = RandomForestClassifier( n_estimators=200, # Builds 200 independent decision trees max_depth=None, # Allows trees to expand until nodes are pure min_samples_split=2, # Minimum samples required to split an internal node min_samples_leaf=1, # Minimum samples required at each leaf node class_weight='balanced', # Adjusts weights inversely proportional to class frequencies random_state=42, # Guarantees deterministic, reproducible results n_jobs=-1 # Utilizes all CPU cores for parallel training )
```

6.4 predict() vs. predict_proba()

- model.predict(X): Returns only the single domain with the highest majority vote (e.g. 'Software Development').
- model.predict_proba(X): Returns an array of continuous probability estimates across all 11 domains (e.g. Software Development: 0.705, AI & ML: 0.155, Data Analytics: 0.055). **CareerCompass AI explicitly uses predict_proba** so that the top-3 ranked alternatives can be presented to the student with transparent match percentages.

6.5 Serialization with Joblib

Training a 200-tree forest takes computational time. By saving the fitted pipeline as a .joblib binary artifact (ml/artifacts/career_rf_pipeline.joblib), the FastAPI server can load the pre-trained weights in **less than 80 milliseconds** during startup, providing sub-second inference times without retraining on every HTTP request.

7. Where Machine Learning Happens in the Codebase

To demystify the system, the following table maps the entire Machine Learning lifecycle to exact files and functions:

Pipeline Stage	File & Function	Input & Output	Exact Implementation Details
1. Multi-Hot Skill Vectorization	ml/preprocessing.py SkillMultiHotEncoder	**In:** Raw skill strings **Out:** Sparse binary matrix	Extracts, normalizes, and alphabetizes all unique skills across the dataset. Implements transform() to assign 1.0 for candidate skills and 0.0 for absent skills.
2. Categorical Preprocessing	ml/preprocessing.py create_preprocessing_pipeline()	**In:** 8 nominal features **Out:** One-hot vectors	Combines SkillMultiHotEncoder and OneHotEncoder(handle_unknown='ignore') inside a unified Scikit-Learn ColumnTransformer.
3. Model Training & Validation	ml/train_model.py train_pipeline()	**In:** Excel dataset **Out:** Fitted Pipeline	Executes 80/20 stratified split, runs 5-fold cross validation, fits the 200-tree Random Forest, and writes metadata to model_metadata.json.
4. Artifact Serialization	ml/train_model.py joblib.dump()	**In:** Pipeline object **Out:** career_rf_pipeline.joblib	Persists the full pipeline containing both the preprocessing transformer and tree ensemble to disk.
5. Model Deserialization	ml/predict.py load_model_artifacts()	**In:** .joblib on disk **Out:** In-memory pipeline	Loads the model into RAM during FastAPI startup event, caching the instance globally to eliminate I/O overhead on prediction requests.
6. Real-Time Inference	ml/predict.py predict_career_domains()	**In:** Candidate profile **Out:** Ranked recommendations	Converts student dictionary into single-row DataFrame, calls pipeline.predict_proba(), ranks top probabilities, and extracts matching skills.
7. REST API Exposure	ml/app.py POST /predict	**In:** JSON HTTP body **Out:** JSON response	Validates request using Pydantic StudentProfileRequest and returns top-3 domain probabilities with skill overlap and disclaimer.

8. Where AI, ML & Rule-Based Logic Are Applied

In technical discussions and project defenses, examiners frequently ask: *'Where is the AI in your system? Is the roadmap generated by an LLM? How is the skill gap calculated?'* It is critical to provide a scientifically precise answer:

Subsystem	Underlying Mechanism	Classification	Technical Justification
Career Domain Prediction	Scikit-Learn Random Forest Classifier (200 trees)	**Machine Learning (AI)**	Learns non-linear boundary mappings from historical training profiles to predict multi-class domain probabilities without manual if-else rules.
Multi-Factor Compatibility	Weighted composite formula (Skills, Interest, Branch, Goals)	**Heuristic Algorithm**	Blends ML confidence with structured constraints (e.g. penalizing branch mismatch while rewarding transferable skills) into a normalized score.
Skill-Gap Identification	Mathematical Set Difference (Core Skills \ Candidate Skills)	**Deterministic Logic**	Directly compares candidate skills against validated industry benchmarks, classifying missing tools into high, medium, and low priority gaps.
Adaptive Phased Roadmap	Parametric Template Engine (4 phases, duration scaled by hours)	**Rules-Based Automation**	Tailors learning phases, milestone objectives, and capstone project deliverables based on identified gaps and study commitments.
Generative AI / LLM	*None in current verified implementation*	**N/A**	**Important:** The current system does NOT call OpenAI, Gemini, or third-party LLMs. Roadmaps are deterministically verified rather than hallucinated.

8.1 The Hierarchy of Artificial Intelligence

• **Artificial Intelligence (AI):** The broad umbrella concept of machines performing tasks that typically require human cognition.
• **Machine Learning (ML):** A specific subset of AI where systems improve performance on a task using data (e.g. our Random Forest Classifier).
• **Deep Learning / Generative AI:** A further subset of ML utilizing multi-layer neural networks or transformers. CareerCompass AI intentionally utilizes tabular Random Forest rather than deep learning because tabular engineering profiles are structured, low-latency, and require deterministic auditability.

9. Backend Architecture & REST API Endpoints

CareerCompass AI runs two distinct local server runtimes: **Express on Port 5000** (Application & User Gateway) and **FastAPI on Port 8000** (Machine Learning Inference Microservice).

Method & URL	Server	Auth Required?	Functionality & Returned Data
GET /api/health	Express (5000)	No (Public)	Returns system status, service name, backend version (2.0.0), and server timestamp.
GET /api/ml-health	Express (5000)	No (Public)	Queries FastAPI's /health to verify model status, active classes (11), and version.
GET /api/domains	Express (5000)	No (Public)	Returns the catalog of 13 engineering career domains with core skills and role titles.
POST /api/auth/register	Express (5000)	No (Public)	Validates email & password, hashes password via bcrypt, creates user record, and issues JWT.
POST /api/auth/login	Express (5000)	No (Public)	Verifies password against stored bcrypt hash; returns JWT and profile completion status.
GET /api/auth/me	Express (5000)	**Yes (Bearer)**	Returns authenticated student identity (id, name, email) and assessment status.
GET /api/student/profile	Express (5000)	**Yes (Bearer)**	Returns the isolated profile and recommendations belonging strictly to the requesting user.
POST /api/recommend	Express (5000)	**Yes (Bearer)**	Main recommendation pipeline: invokes ML microservice, computes roadmaps, and saves results.
PUT /api/student/roadmap	Express (5000)	**Yes (Bearer)**	Updates task milestone status (not_started, in_progress, completed).
POST /predict	FastAPI (8000)	Internal / Open	Microservice endpoint executing predict_proba() on candidate feature vectors.

9.1 Sample REST JSON Interaction (Inference Payload)

```
// POST http://127.0.0.1:8000/predict // REQUEST PAYLOAD: { "student_branch": "Computer Science Engineering", "study_year": "3rd Year", "current_skills": ["Python", "SQL", "Machine Learning", "pandas"], "primary_interest": "AI, prediction and experimentation", "work_style_preference": "analytical problem-solving", "career_aspiration": "AI Research or ML Engineer" } // RESPONSE PAYLOAD (FastAPI ML Service): { "success": true, "model_type": "RandomForestClassifier", "model_version": "1.0.0", "total_classes_evaluated": 11, "recommendations": [ { "domain": "AI & Machine Learning", "score": 0.815, "estimated_probability_percent": 81.5, "matching_skills": ["Python", "Machine Learning", "pandas"], "skill_gaps": ["Linear algebra", "scikit-learn", "NumPy"], "explanation": "Direct foundation in Python, pandas; aligned with CSE curriculum" } ] }
```

10. Training and Evaluation Results

10.1 Experimental Protocol & Validation Setup

The model training pipeline (ml/train_model.py) enforces rigorous machine learning validation procedures: an 80/20 stratified train/test split (440 training samples, 110 held-out testing samples) and Stratified 5-Fold Cross Validation across the training set to preserve equal class representations.

Metric / Parameter	Observed Value	Interpretation & Significance
Total Dataset Size	550 student profiles	Complete synthetic dataset from Excel workbook.
Training Set Size	440 samples (80%)	Used for fitting both preprocessing transformers and tree estimators.
Testing Set Size	110 samples (20%)	Held-out test set completely unseen during model fitting.
Cross-Validation Accuracy	1.0000 (100.0%) ± 0.00	Stratified 5-Fold CV mean accuracy across all 5 folds.
Held-Out Test Accuracy	1.0000 (100.0%)	Exact accuracy achieved on the 110 held-out test samples.
Macro Avg Precision / Recall	1.0000 / 1.0000	Zero false positives or false negatives across all 11 classes.
Macro Avg F1-Score	1.0000 (100.0%)	Harmonic mean of precision and recall indicates perfect classification on this data.

10.2 Why Did the Model Achieve 100% Accuracy? (Critical Exam Topic)

In a viva or project defense, achieving 100% accuracy immediately invites scrutiny: *'Is there data leakage? Did the model memorize the data?'*

The Technical Explanation:
1. **Zero Target Leakage:** All 11 post-recommendation columns (e.g. Suggested_Career_Path, Roadmap_Step_1..5) were explicitly purged before training. The model only trained on independent student attributes.
2. **Deterministic Synthetic Generation:** In the synthetic Excel dataset, combinations of Primary_Interest and Current_Skills are strongly correlated with the target label (e.g. every student with interest *'buildings, structures and digital construction'* maps deterministically to *Civil Design & BIM*; every student with *'hardware, sensors and connected devices'* maps to *Embedded Systems & IoT*). The feature importance report confirms this: Primary_Interest accounts for over 60% of total Gini impurity reduction.
3. **Real-World Recommendation:** While 100% accuracy proves that the Random Forest successfully learned the synthetic generation rules with zero error, real engineering students have ambiguous, overlapping, and contradictory interests. Deploying the system to real students will introduce noise, which is why future iterations should gather human empirical feedback.

11. Skill-Gap Analysis & Roadmap Generation

11.1 Skill-Gap Analysis Algorithm

Once a target domain is identified, the system conducts a multi-tier skill discrepancy analysis (implemented in server/recommendationEngine.js). The domain's catalog benchmark skills (S_{domain}) are compared against the candidate's verified skills ($S_{student}$) using set theory:

- **Matched Skills:** $S_{student} \cap S_{domain}$ — Technical tools the student already possesses (e.g. Python, SQL).
- **Skills to Improve:** Skills in $S_{student}$ marked as 'Beginner' that are critical to the domain.
- **High-Priority Gaps:** $S_{domain} \setminus S_{student}$ — Core industry tools completely absent from the student's profile (e.g. Power BI, Statistics, Tableau for Data Analytics).

11.2 Illustrative Example: Student Targeting Data Analytics & BI

Candidate Skills: Python (Intermediate), SQL (Intermediate), HTML/CSS (Beginner)
Target Domain Core Skills: Python, SQL, Excel, Statistics, Data analysis, Power BI, Tableau

Classification	Skills Identified	Actionable System Guidance
Verified Strengths	Python, SQL	Marked as foundational prerequisites already completed. Student can skip basic programming tutorials.
Skills to Improve	HTML/CSS	Low priority for analytics; deprioritized in roadmap to avoid cognitive overload.
Critical Skill Gaps	Excel, Statistics, Power BI, Tableau	Inserted as core objectives into Phase 1 and Phase 2 of the personalized curriculum.

11.3 Adaptive Phased Roadmap Structure

The roadmap engine (generateAdaptiveRoadmap) generates four progressive milestones scaled dynamically by the student's weekly study commitment:

- **Phase 1: Foundation & Tooling (2-3 Weeks):** Consolidates fundamental logic, sets up local IDE, Git, and environments. Deliverable: Working local development workspace.
- **Phase 2: Core Domain Deep-Dive (4-6 Weeks):** Bridges high-priority skill gaps through structured drills. Deliverable: Small-scale domain prototypes.
- **Phase 3: Portfolio Capstone Project (4-5 Weeks):** Builds an end-to-end production-quality application with unit tests and README. Deliverable: Public GitHub repository with live demo.
- **Phase 4: Interview & Placement Readiness (2-4 Weeks):** ATS-optimized resume formatting, technical question drills, and mock interview practice. Deliverable: Polished portfolio website and active applications.

12. Authentication System & Data Isolation

12.1 Security & Authentication Architecture

CareerCompass AI implements industrial-standard security protocols to protect student identities and ensure per-student data privacy:

- **Bcrypt Password Hashing:** Plaintext passwords are never stored. Upon registration, bcrypt.hash(password, 10) applies 10 salt rounds to generate a cryptographically irreversible 60-character hash (e.g. $2b$10$...).
- **JSON Web Tokens (JWT):** Upon valid login, the server generates a signed cryptographic token containing { userId, email, name } with a 7-day expiration. The frontend stores this token in browser local storage and passes it in the Authorization: Bearer <token> header on all protected requests.
- **Middleware Route Protection:** The authenticateToken middleware intercepts requests to /api/student/* and /api/recommend. If the token is missing or invalid, the server immediately rejects the request with HTTP 401 Unauthorized.

12.2 Per-Student Data Isolation Model

To prevent one student from viewing or overwriting another student's assessment, data is strictly partitioned in file storage by userId:

Storage Entity	File Location	Data Structure & Isolation Mechanism
User Accounts	server/data/users.json	Dictionary keyed by userId storing { id, name, email, passwordHash, createdAt }. Plaintext passwords never exist.
Student Profiles & Progress	server/data/saved_profiles.json	Dictionary keyed strictly by userId. Endpoints look up db[req.user.userId]. A student cannot access any other key.
Password Resets	server/data/password_resets.json	Dictionary keyed by secure reset tokens with 1-hour expiration timestamps.
Career Catalog	server/data/career_domains.json	Static platform catalog detailing 13 engineering domains, potential roles, and benchmark skills.

12.3 Name Synchronization & Greeting Bug Fix

During early testing, an issue occurred where a newly logged-in student (*Vidhi Joshi*) saw the greeting *'Welcome back, Aarav Sharma!'* on their dashboard. Investigation revealed that the browser's local storage held a stale un-scoped cache key (careercompass_profile) left over from an earlier demo test. The issue was resolved across three layers: (1) Client storage was scoped to careercompass_profile_${userId}; (2) The Express backend now forcefully overrides any profile name with the authenticated account name req.user.name; and (3) The React dashboard greeting reads directly from userName || profile.name.

13. Project File Structure & Code Map

The following guide outlines the essential files in the repository and indicates the recommended inspection order for beginners:

File / Folder	Inspection Order	Architectural Responsibility
client/src/App.tsx	**#1** (Start Here)	Central frontend orchestrator managing views (auth, assessment, dashboard), user session, and modal states.
client/src/components/AuthPage.tsx	#2	Authentication interface: Sign In, Sign Up, and Password Reset screens with password strength validation.
client/src/components/AssessmentWizard.tsx	#3	Step-by-step interactive student assessment capturing branch, year, skills, proficiencies, and career goals.
client/src/components/Dashboard.tsx	#4	Personalized career dashboard rendering top-3 match cards, skill radar chart, and phased roadmap milestones.
server/server.js	**#5** (Backend Gateway)	Primary Node/Express server hosting REST APIs, JWT authentication route guards, and orchestrating ML calls.
server/recommendationEngine.js	#6	Multi-factor heuristic scoring engine and dynamic 4-phase adaptive roadmap generator.
server/auth.js	#7	Security module handling bcrypt password hashing, token generation, and the authenticateToken middleware.
ml/app.py	**#8** (ML Service)	FastAPI Python application exposing /predict and /health endpoints for Random Forest inference.
ml/train_model.py	#9	Model training pipeline script executing data loading, stratified CV, Random Forest fitting, and joblib export.
ml/preprocessing.py	#10	Feature transformers: SkillMultiHotEncoder and categorical OneHotEncoder in a ColumnTransformer.
ml/artifacts/*.joblib	#11	Trained scikit-learn model artifact binary and accompanying model_metadata.json.
test_complete_platform.js	#12	Automated end-to-end test suite testing health, authentication, password hashing, isolation, and recommendations.

14. How to Run the Project Locally

Follow these exact step-by-step instructions to run CareerCompass AI on a local computer:

Step 1: Prerequisites

• **Python:** Python 3.10+ (Python 3.14 verified in this workspace).
• **Node.js:** Node.js v18+ (Node v24.19 verified).
• **Browser:** Modern Chromium, Firefox, or Edge browser.

Step 2: Install Dependencies

```
# 1. Install root, client, and server Node packages npm run install:all # 2. Install Python ML requirements py -m pip install fastapi uvicorn scikit-learn pandas numpy openpyxl joblib pydantic
```

Step 3: Start the FastAPI Machine Learning Service (Terminal 1)

```
# Start FastAPI ML Microservice on Port 8000 py -m uvicorn ml.app:app --host 127.0.0.1 --port 8000 # Output should indicate: 'Application startup complete.' and 'Uvicorn running on http://127.0.0.1:8000'
```

Step 4: Start the Web Application & Backend (Terminal 2)

```
# Build the React client production bundle cd client && npm run build && cd .. # Start the Express server on Port 5000 node server/server.js # Output: 'CareerCompass AI Server running on port 5000'
```

Step 5: Access the Platform in Browser

Open **http://localhost:5000**. You will be greeted with the authentication screen. Sign up with a new student account or log in.

Step 6: Run Automated Verification Tests

```
# Run complete end-to-end test suite (14 checks) node test_complete_platform.js # Run ML unit tests py ml/test_ml_pipeline.py
```

15. Common Errors, Diagnosis & Troubleshooting

Symptom / Error Message	Root Cause	Verification & Fix
FastAPI connection refused / 503 Service Unavailable	FastAPI ML microservice is not running on port 8000.	Open Terminal 1 and start uvicorn: py -m uvicorn ml.app:app --host 127.0.0.1 --port 8000. Verify at http://127.0.0.1:8000/health.
Port 5000 already in use (EADDRINUSE)	A previous instance of Node is still running in the background.	Kill the stale process via PowerShell: Get-Process -Id (Get-NetTCPConnection -LocalPort 5000).OwningProcess | Stop-Process -Force.
Model artifact not found / FileNotFoundError	The .joblib artifact has not been trained or was moved.	Run the training script to regenerate the model: py ml/train_model.py. Verify that career_rf_pipeline.joblib exists in ml/artifacts/.
HTTP 401 Unauthorized on Dashboard	Session token has expired or is absent from local storage.	Click Log In and re-authenticate. The client will store a fresh JWT in local storage.
Changes in React components not showing in browser	Express serves static production assets from client/dist.	Run cd client && npm run build to compile TypeScript to client/dist, then refresh the browser (Ctrl + F5).
Bcrypt build errors during npm install	Native C++ node-gyp build dependencies missing.	The project intentionally uses **bcryptjs** (pure JavaScript implementation) to guarantee seamless installation on Windows without C++ compilers.

16. Project Limitations & Future Scope

16.1 Honest Project Limitations

Every credible engineering project must acknowledge its boundaries. CareerCompass AI currently operates under the following known technical constraints:

- **Synthetic Training Data:** The 550 training profiles were generated using rule-based templates. While excellent for prototyping the pipeline, it does not capture the subtle nuances or contradictions of real human career decisions.
- **Domain Class Coverage:** The dataset contains 11 target classes, requiring the other 2 catalog domains (Chemical and AR/VR) to rely on heuristic fallback rules.
- **Static Skill Catalog:** Technical frameworks evolve rapidly. New tools (e.g. Next.js 15, Mojo, LangChain) must currently be added manually to the metadata catalog.
- **Local File-Based Storage:** While optimal for local execution and zero-dependency testing, production deployment at institutional scale would benefit from a relational database (PostgreSQL) with connection pooling.

16.2 Strategic Future Scope

The platform was architected to support future evolutionary enhancements:

- **Human-in-the-Loop Feedback:** Introduce post-placement surveys where graduated students validate their career satisfaction, continuously retraining the Random Forest model on verified outcomes.
- **Conversational AI Assistant (LLM Integration):** Integrate a secure Large Language Model (e.g. Gemini 1.5 Flash) strictly as an interactive conversational tutor for explaining roadmap milestone concepts.
- **GitHub Portfolio Analyzer:** Allow students to connect their GitHub profile, automatically extracting their code repositories, commit activity, and language frequency to pre-fill their skills assessment.
- **Automated ATS Resume Parser:** Enable PDF resume uploads that automatically parse technical skills, academic performance, and branch background.

17. Comprehensive Viva & Presentation Preparation

17.1 The 1-Minute Elevator Pitch

'CareerCompass AI is a full-stack, AI-powered career recommendation platform specifically designed for engineering college students. Students often learn multiple programming languages and tools but struggle to identify which industry domain fits them best. Our platform assesses their technical skills, branch, and preferences, processes them through a trained Random Forest Classifier pipeline, ranks the top 3 matching career domains with transparent explainability, and automatically generates an adaptive, 4-phase milestone roadmap to help them bridge skill gaps and become placement-ready.'

17.2 The 3-Minute Comprehensive Presentation Structure

1. Problem Statement (45s): Introduce the challenge of career ambiguity in engineering colleges. Highlight that students pick online courses randomly without understanding domain requirements.
2. Machine Learning Pipeline (60s): Explain the Random Forest Classifier. Detail how skills are transformed via custom Multi-Hot encoding, how 200 trees vote to produce continuous class probabilities, and how the model was evaluated.
3. System Demo & Architecture (45s): Show the React frontend connecting to Express on Port 5000 and FastAPI on Port 8000. Show user registration with Bcrypt and JWT, and display the personalized roadmap.
4. Results & Future Scope (30s): Conclude with the importance of explainable AI over black-box predictions and outline future plans for real-world feedback loops.

17.3 Top 25 Viva Questions with Beginner-Friendly Answers

Q1: What is the main objective of CareerCompass AI?

To help engineering students discover optimal career domains based on their existing technical skills, branch, and working preferences, providing an explainable roadmap to bridge skill gaps.

Q2: Why did you choose Random Forest instead of a Single Decision Tree?

A single decision tree easily overfits and has high variance. A Random Forest combines 200 diverse trees using bagging, voting across the ensemble to yield higher generalization and reliable probability distributions.

Q3: Why Random Forest instead of Deep Learning / Neural Networks?

Tabular student data with categorical and multi-hot features is best handled by tree-based ensembles. Neural networks require massive datasets and act as black boxes, whereas Random Forests provide fast inference and feature importance explainability.

Q4: What is Supervised Learning?

A machine learning paradigm where the model learns from labeled training examples consisting of input feature vectors X and corresponding target labels y.

Q5: What is Classification?

Predicting a discrete category or class label (e.g. 'Software Development' vs. 'Cybersecurity') rather than a continuous numerical value (which would be regression).

Q6: What is a Feature in Machine Learning?

An individual measurable property or characteristic of a student profile used as an input to the model (e.g. Current_Skills, Student_Branch, Primary_Interest).

Q7: What is the Target Variable in your project?

'Recommended_Domain'—the ground-truth career classification label present in the training dataset.

Q8: What does predict_proba() do?

Returns the percentage of decision trees in the forest that voted for each class, representing the model's confidence or probability distribution across all domains.

Q9: What does 100% accuracy on your synthetic dataset mean?

It indicates that the Random Forest successfully learned the deterministic relationships present in the synthetic dataset without error. However, we acknowledge that real-world student data will require empirical validation.

Q10: What is Data Leakage and how did you prevent it?

Data leakage occurs when information from outside the training dataset (or derived from the target) contaminates the model. We prevented it by explicitly dropping post-recommendation columns (e.g. Suggested_Career_Path, Roadmap_Steps) before training.

Q11: Why do you use Stratified K-Fold Cross Validation?

Stratified K-Fold splits the dataset into folds while ensuring every fold has the exact same percentage of each domain class, preventing minority class starvation during validation.

Q12: What is Joblib and why is it used?

Joblib is a Python library optimized for serializing large NumPy arrays and Scikit-Learn pipelines to disk, allowing instant model loading during server startup.

Q13: What is FastAPI and why is it used?

FastAPI is a modern, high-performance Python web framework used to expose our trained model over a REST API (port 8000) with automatic Pydantic request validation.

Q14: How does the React frontend talk to the FastAPI model?

The React frontend sends requests to the Express gateway (port 5000), which verifies the user's token and forward-proxies the profile to FastAPI (port 8000) via HTTP JSON.

Q15: Is your learning roadmap generated by AI?

No. The domain recommendation uses Machine Learning, but the roadmap is generated by a deterministic parametric rules engine that tailors 4 milestone phases based on calculated skill gaps and study hours.

Q16: How is the Skill Gap calculated?

Using set difference: Domain Core Skills minus Student Possessed Skills. Missing skills are prioritized into high, medium, and low urgency.

Q17: Why do you have 11 classes in the dataset but 13 domains in the catalog?

The 11 core classes have synthetic ML training examples. Two newer disciplines (Chemical and AR/VR) use verified heuristic compatibility rules as a fallback.

Q18: What is Bcrypt and why is it used?

Bcrypt is an adaptive cryptographic password hashing function. It incorporates random salts (10 rounds) to protect passwords against rainbow table and brute-force attacks.

Q19: What is a JSON Web Token (JWT)?

A compact, URL-safe token containing cryptographically signed user claims, allowing stateless authentication across HTTP requests.

Q20: How do you prevent one student from viewing another student's data?

All profile reads and writes in server/data/saved_profiles.json are keyed strictly by the verified req.user.userId extracted from the validated JWT token.

Q21: What is the purpose of the SkillMultiHotEncoder?

Candidate skills are lists of strings. Standard encoders cannot parse comma-separated lists. The custom encoder creates a fixed-width binary vector where 1 indicates skill presence and 0 indicates absence.

Q22: What role does Tailwind CSS play?

Tailwind provides utility classes for our custom SaaS visual design system, delivering responsive layouts, consistent navy/blue color tokens, and clean typography.

Q23: What is the difference between n_estimators and max_depth in Random Forest?

n_estimators is the total count of decision trees in the forest (200 in our model). max_depth is the maximum depth allowed for each individual tree (None allows full expansion).

Q24: What happens if a student inputs a skill not present in training data?

The SkillMultiHotEncoder safely ignores unseen skills during transformation (yielding zeros), preventing runtime index errors or server crashes.

Q25: What is the single biggest limitation of the current project?

The reliance on synthetic student training data rather than empirical, multi-year post-graduation career placement outcomes.

18. Glossary of Technical Terms

Bagging (Bootstrap Aggregation)	An ensemble meta-algorithm where multiple models are trained on random subsets of the dataset with replacement, and their outputs are averaged or voted on to reduce variance.
Bcrypt	A password hashing function based on the Blowfish cipher, designed to remain computationally expensive over time to resist hardware-accelerated attacks.
ColumnTransformer	A Scikit-Learn utility that applies different preprocessing transformers to different subsets of columns (e.g. multi-hot encoding for skills, one-hot for branch).
Cross-Validation	A statistical technique for assessing model generalization by partitioning data into multiple subsets and rotating training and testing roles.
Decision Tree	A flowchart-like tree structure where internal nodes represent tests on features, branches represent outcomes, and leaf nodes represent final class labels.
Explainable AI (XAI)	Methods and techniques in artificial intelligence that allow human experts to understand and trace how a machine learning model arrived at its predictions.
FastAPI	A modern, asynchronous Python web framework for building APIs, leveraging Python type hints and Pydantic for high performance and automatic documentation.
Gini Impurity	A mathematical metric measuring the probability of an element being incorrectly classified when chosen at random. Random Forests use it to pick optimal feature splits.
JWT (JSON Web Token)	An open industry standard (RFC 7519) representing claims securely between two parties, typically used for stateless authentication.
Multi-Hot Encoding	A vector representation where multiple entries in a binary array can be set to 1 simultaneously, representing the presence of multiple items (e.g. a student knowing Python AND SQL).
One-Hot Encoding	A vector representation for single-choice categorical variables where exactly one entry is set to 1 and all others are 0.
Overfitting	A modeling error occurring when an algorithm learns noise and eccentricities of the training data so closely that it fails to generalize to new, unseen data.
Pydantic	A data parsing and validation library for Python that uses type annotations to enforce schema integrity and data serialization.
Random Forest	An ensemble learning method for classification and regression that operates by constructing a multitude of decision trees at training time and outputting class majority votes.
REST (Representational State Transfer)	An architectural style for network applications relying on stateless client-server communication using standard HTTP verbs (GET, POST, PUT, DELETE).
Scikit-Learn	An open-source Python library providing simple and efficient tools for predictive data analysis and machine learning.
Set Difference	A mathematical operation on two sets A and B that yields the elements belonging to A but not to B (used in our project to calculate skill gaps).
Supervised Learning	Machine learning where the algorithm is supplied with labeled training pairs consisting of input features and desired target outputs.
Target Leakage	An error where information that will not be available at prediction time is inadvertently included in the training features, leading to falsely optimistic metrics.
Vite	A frontend build tool that leverages native browser ES modules to deliver ultra-fast hot module replacement during development and optimized Rollup/esbuild production bundles.

19. Final Quick-Revision Sheet & System Summary

Key Dimension	Summary Specification
Project Name	CareerCompass AI
Core Mission	Intelligent, explainable career domain discovery and adaptive phased roadmaps for engineering students.
Target Users	Engineering college students (1st through 4th Year) across Computing, Electronics, Electrical, Mechanical, Civil & Chemical disciplines.
Frontend Stack	React 19, TypeScript, Tailwind CSS v4, Lucide Icons, Vite (Served at http://localhost:5000).
Backend Stack	Node.js, Express.js 5, Bcryptjs password hashing, JWT authentication (Port 5000).
ML Stack & Microservice	Python 3.14, FastAPI, Uvicorn, Scikit-Learn, Pandas, NumPy, Joblib (Served at http://127.0.0.1:8000).
Machine Learning Algorithm	RandomForestClassifier(n_estimators=200, class_weight='balanced', random_state=42).
Model Artifact	ml/artifacts/career_rf_pipeline.joblib (Includes custom SkillMultiHotEncoder & categorical OneHotEncoder).
Training Dataset	Engineering_Career_Recommendation_Dataset_550.xlsx (550 synthetic engineering profiles across 11 classes).
Target Variable	Recommended_Domain (Categorical label predicting ground-truth domain).
Inference Mechanism	predict_proba() continuous class probabilities ranked into Top 3 recommended career domains.
Roadmap Engine	Dynamic 4-phase parametric template rules engine scaled by weekly hours and set-difference skill gaps.
Security & Data Model	Bcrypt salted hashing (10 rounds), signed JWT tokens, isolated profile storage in server/data/saved_profiles.json.
Evaluation Metric	100% Stratified 5-Fold Cross-Validation accuracy on synthetic benchmark data; zero target leakage.
Verified Status	End-to-End verified: all 14 platform tests and 7 ML unit tests passing cleanly.

Final Takeaway for Your Presentation:

*CareerCompass AI proves how modern machine learning and full-stack web engineering can merge to solve genuine educational challenges. By combining **supervised Random Forest classification for domain prediction** with **deterministic explainable logic for skill gaps and roadmaps**, the platform provides students with transparent, actionable, and trustworthy guidance without relying on opaque black-box systems.*